

Video link:

https://mediaspace.msu.edu/media/CompanyC-Group9-Database%20Video/1_rwxh5h6q

Datum - Ian Wettlaufer

Hi, I am going to be walking through the Datum class I wrote. It's basically a flexible value container that can hold a double, string or bool all in one type. It can be thought of like a dynamic variable, similar to how variables are handled in python or JavaScript.

The idea of the class is simple, you can construct a Datum with any of the supported types, and it can handle the rest. You don't need to worry about what the type was originally as it is remembered by the Datum class. The most useful part of this class is the conversion methods. You can ask any Datum object to give you its value as a double, a bool, or a string regardless of the original type. A Datum holding the string "3.14", when AsDouble() is called on it will automatically try to convert the underlying value into a double and give you the number. Calling AsBool() on a number returns false if it's 0, and true for all else. Same for strings, empty is false, non-empty will return true. Additionally you can do math directly on Datum objects such as addition, subtraction, multiplication and division without the need to convert the values manually. Division by zero yields a NaN value allowing the program to fail gracefully instead of crashing. The programmer can simply check the value using std::isnan() to know if there was an illegal operation that occurred. Comparison Operators are also supported, equals, not equals, less than, greater than, <= and >=. Comparing a string to a double will attempt to convert and compare numerically, whereas comparing two non-numeric strings will check lexicographically. Main considerations when using this class - When you try to do math on a non-numeric string, NaN is returned, so it's worth checking using the built in IsString(), IsDouble or IsBool() first if you are unsure what you are working with, or checking afterward that std::isnan() returns false before continuing on. This class is simply meant to make value handling more flexible without all of the manual type juggling.

StringCompressor - Krist Veseli

Hello everybody, I will be walking you through one of Group 9's core components: the StringCompressor class.

Every time an agent moves through a world, or a player sends a message, we are generating data. StringCompressor keeps our database lean by compressing this data before storage.

The StringCompressor class is responsible for compressing and decompressing string data using the LZW algorithm, which is a proven lossless compression technique that works well on repetitive text. LZW works by building a dictionary of patterns as it reads through the string. It starts with all 256 basic ASCII characters, and as it encounters repeated sequences, it adds them to the dictionary and replaces them with a single code. This means the longer and more repetitive the data, the more efficient the compression becomes.

For example, compressing "TOBEORNOTTOBEORNOT" recognizes patterns like "TO", "BE", and "OR", reducing 19 characters down to 13 codes - about 30% space saving.

The class provides two core operations: Compressing, which takes a string and encodes it into a compact sequence of codes, and Decompressing, which restores the original string. It also includes byte serialization for raw storage, a magic number header to identify compressed data, and a GetCompressionRatio function to measure space saving.

StringCompressor will be primarily used by the Serializer class. After the Serializer converts data of any type into a string, StringCompressor compresses it before storage or messaging. It also works alongside StringDiff, which handles a related but different task - encoding only the differences between two strings for efficient updates.

StringDiff - Andrew Shilman

Hi there, I'm Andrew Shilman and I'll be walking you through Group 9's StringDiff class.

The StringDiff class is responsible for essentially finding, applying, and of course encoding and decoding smaller differences in code for easy, quick, and efficient transportation of these changes. So imagine this, you have a book with 10,000 words and only one word was changed. Would it be more efficient to delete everything and re-write this book with the one word changed, or to simply highlight the change and only change that one little portion. That is what StringDiff class does.

So the StringDiff class has 4 main functions, MakeDiff, ApplyDiff, EncodeDiff, and DecodeDiff. To start off I will mention things called a "Diff" or a "patch" sometimes. These are just instructions so we know the small changes. A diff consists of 5 things, base_hash, base_length, prefix_length, suffix_length, and the replacement. The base_hash and base_length are just signature information to utilize later on, prefix_length is the length of all the text that is the same before the change, suffix_length is that but after the change, and replacement_len and replacement is the change to be made.

Starting off our MakeDiff class takes 2 strings, the original, and the updated, and breaks it down to return a Diff of the changes made, with all the information as saved above. ApplyDiff takes this Diff or patch, verifies everything is okay, strips it down and all it does is return the updated version. ApplyDiff takes in the starter string, and the patch or Diff, which once again is just instructions, and returns the result.

Though this can be a lot of information having to continuously get these Diffs so I made an Encode and Decode. The encode takes in a Diff and encodes it into an easy and readable format, such as HASH | BASE_LEN | PREFIX_LEN, etc.... So it can be transported over as a simple string compared to just a Diff. Decode takes this in and breaks it down and returns it as a Diff.

Serializer - Yashvasin Siddavarapu

Hey everyone, in this video I'm going to walk you through the Serializer class. The Serializer takes C++ objects, converts them into a tagged text format we can store or transmit, and converts them back. It's header-only, you just include `Serializer.hpp` and you're good to go, no extra linking.

Every value gets a type tag up front so the deserializer knows what it's reading. Ints get `i`, doubles get `d`, bools get `b`, chars get `c`. Strings use a length prefix — so `s:5:hello;` — which means colons and semicolons inside the string are totally fine. Vectors and maps store their size and then serialize each element recursively.

Using it is simple. You create a Serializer instance, then call `Serialize` with any supported type. Ints, doubles, bools, chars, and strings each produce their tagged output. For deserialization, every method returns a `std::optional`, so you always check if it has a value before using it. That means bad data gives you an empty optional instead of a crash.

Vectors and maps work the same way. `Serialize` takes the container directly. To deserialize, you call `DeserializeVector` or `DeserializeMap` with the element types as template parameters. In the example, we round-trip a vector of ints and a map of string-to-int and get back exactly what we put in.

Now the most important part: custom types. The Serializer doesn't magically know how to handle your own structs. You have to teach it by registering the type with two lambdas: one that serializes your object to a string, and one that deserializes the string back into your object.

Here's the serialize lambda for our `Agent` struct. It takes a `const Agent` reference and returns a string. All it does is call `s.Serialize` on each field — `name`, `health`, `level` — and concatenate the results. So for an `Agent` named `Steve` with 100 health and level 7, it produces `s:5:Steve;d:100;i:7;` — each field tagged and combined.

The deserialize lambda is the reverse. It takes the serialized string and returns an optional `Agent`. The key idea is the `pos` variable. It starts at zero, and every time we call `DeserializeAt`, `pos` moves forward past whatever field we just read. First we read the name and `pos` advances past `s:5:Steve;`. Then we read health and `pos` advances past `d:100;`. Then we read level and `pos` advances past `i:7;`. We check that all three succeeded, assign the values to our `Agent` struct, and return it. If anything fails, we return `nullopt`.

DataGrid - Mitchell McAuley

The DataGrid class is a data structure that compactly stores information about the world in Datum objects. A DataGrid is able to support objects of any type with quick lookup, read and write operations. Its constructor asks you to specify its dimensions, but they can be dynamically expanded. You can also parse the DataGrid using the built in Iterator class. The functions listed implement basic functionality for managing a matrix of objects, like isolating rows or columns, indexing into specific coordinates, searching for keys, and more. I expect this class to be expanded as the project's scope becomes fully realized, but I'll most likely add support to allow users to store information in column-major form, a Transpose function for reordering the array, as well as Save/Load functions for converting data to and from a save file like XML. I can adopt many other functions similar to those in Python's "numpy" module. The underlying data structure is a vector of vectors of datum objects.